Proyecto Integrador: Plataforma SaaS Multitenant

Proyecto individual · Duración: 1 semana · Se permite el uso de IA como apoyo, pero se evalúa con sustentación oral

1. Objetivo

Construir una plataforma **SaaS multitenant**: una sola aplicación que atiende a muchos clientes (tenants), donde cada cliente ve únicamente sus propios datos. El proyecto reúne todo lo visto en el curso: autenticación, ORM, contenedores, CI/CD, estrategias de renderizado, y dos temas de robustez que son obligatorios: **idempotencia** y **rate limiting**.

No se evalúa la cantidad de funcionalidades, sino qué tan bien está resuelta la arquitectura. Es preferible un dominio pequeño y bien hecho que uno grande a medias.

2. ¿Qué es un SaaS multitenant y qué nichos lo necesitan?

Un SaaS (Software as a Service) es un producto que los clientes usan por suscripción, sin instalar nada. Es **multitenant** cuando un mismo sistema sirve a muchas organizaciones (los tenants) y mantiene sus datos aislados. Cada tenant tiene sus propios usuarios y sus propios registros.

Este modelo aparece en cualquier negocio pequeño o mediano que necesita software de gestión pero no puede pagar uno hecho a la medida. Algunos nichos de ejemplo:

- **Clínicas y consultorios:** el tenant es la clínica. Entidades posibles: pacientes, citas, profesionales.
- **Gimnasios y estudios de entrenamiento:** el tenant es el gimnasio. Entidades: socios, planes de membresía, asistencias.
- **Restaurantes y cafeterías:** el tenant es el local. Entidades: productos del menú, pedidos, mesas.
- **Academias e institutos:** el tenant es la institución. Entidades: estudiantes, cursos, inscripciones.
- **Barberías y salones de belleza:** el tenant es el salón. Entidades: servicios, citas, clientes.
- **Inmobiliarias:** el tenant es la agencia. Entidades: propiedades, clientes interesados, visitas.
- **Talleres mecánicos:** el tenant es el taller. Entidades: vehículos, órdenes de servicio, repuestos.

- **Veterinarias:** el tenant es la veterinaria. Entidades: mascotas, propietarios, consultas.
- **Empresas de transporte y logística:** el tenant es la empresa. Entidades: vehículos, rutas, envíos.
- **Despachos de abogados:** el tenant es el despacho. Entidades: casos, clientes, documentos.
- **Coworking y alquiler de espacios:** el tenant es el espacio. Entidades: salas, reservas, miembros.
- **Tiendas en línea pequeñas:** el tenant es la tienda. Entidades: productos, inventario, pedidos.
- **Organizadores de eventos:** el tenant es la productora. Entidades: eventos, asistentes, entradas.
- **Gestión de cumplimiento (GRC):** el tenant es la empresa. Entidades: riesgos, controles, auditorías.

El estudiante puede elegir uno de estos nichos u otro propio, pero debe informarlo al docente para validar que el alcance sea adecuado.

3. Modalidades permitidas

Cada estudiante elige **una** de estas opciones:

- **Opción A:** un backend en el lenguaje y framework de su preferencia (Node, .NET, Python, PHP, Java, etc.) más un frontend en React.
- **Opción B:** Next.js como fullstack, con API y frontend dentro del mismo proyecto.

Los requisitos funcionales son los mismos en ambas opciones. Cambia únicamente cómo se organiza el código y qué estrategias de renderizado aplican (ver sección 4.9).

4. Requisitos obligatorios

4.1 Multitenancy con un schema por tenant

- La base de datos es PostgreSQL. Se permite usar Supabase o un Postgres propio en Docker.
- Debe existir un schema compartido (por ejemplo `public`) con la información global: tabla de tenants y de usuarios.
- Cada tenant tiene **su propio schema** con las tablas de su dominio.
- Al registrarse un tenant nuevo, el sistema debe **crear su schema y sus tablas de forma automática**.

- En cada petición, el sistema debe identificar a qué tenant pertenece el usuario y trabajar únicamente sobre su schema.
- Un usuario de un tenant **nunca** debe poder leer o modificar datos de otro. Esto se debe poder demostrar en la sustentación.

4.2 Autenticación y sesión

- Registro e inicio de sesión.
- Sesión mediante **JWT guardado en una cookie httpOnly**, no en `localStorage`.
- Rutas protegidas en el backend y en el frontend.
- El token debe incluir o permitir resolver el tenant del usuario.

4.3 Dominio de negocio

- Entre 2 y 3 entidades por tenant, con operaciones CRUD completas.
- Validación de datos de entrada en el backend.
- Uso de un ORM o query builder (Prisma en el caso de Node y Next; el equivalente según el lenguaje elegido).

4.4 Idempotencia (obligatorio)

- Al menos **dos operaciones de escritura** deben ser idempotentes. Por ejemplo: crear un tenant, registrar un pedido, una reserva o un pago.
- Se debe usar una clave de idempotencia enviada por el cliente (por ejemplo, un header `Idempotency-Key`).
- Si se repite la misma petición con la misma clave, el sistema devuelve el resultado original y **no vuelve a ejecutar la operación**.
- Se debe demostrar con una prueba: enviar la misma petición dos veces y verificar que solo se creó un registro.
- El estudiante debe explicar dónde guarda las claves y cuánto tiempo las conserva.

4.5 Rate limiting (obligatorio)

- Debe existir limitación de peticiones al menos en el **inicio de sesión** y en los **endpoints de creación**.
- Al excederse el límite, la respuesta debe ser **429** e incluir información de cuándo reintentar (header `Retry-After`).
- El estudiante debe explicar qué estrategia usó (ventana fija, ventana deslizante, token bucket) y por qué la eligió.

- Se valora que el límite se aplique por usuario o por tenant y no solo por IP, y que se justifique la decisión.

4.6 Docker

- Un `docker-compose.yml` que levante todo el proyecto con un solo comando.
- Archivo `.env.example` con las variables necesarias, sin secretos reales.

4.7 CI/CD

- Un pipeline que, como mínimo, instale dependencias, ejecute el lint o el build y falle si algo está mal.
- Se valora que también ejecute pruebas y construya la imagen de Docker.

4.8 Documentación

Un `README.md` que incluya:

- Qué nicho eligió y qué hace el sistema.
- Un diagrama o descripción de la arquitectura multitenant.
- Cómo levantar el proyecto.
- Cómo se resuelve el tenant en cada petición.
- Cómo se implementó la idempotencia y el rate limiting, y por qué se tomaron esas decisiones.

4.9 Estrategias de renderizado

- **Opción B (Next):** usar al menos **dos** estrategias distintas entre SSR, CSR y SSG, cada una en una pantalla donde tenga sentido, y justificar la elección. Ejemplo: una página pública de presentación con SSG, y un panel privado con CSR o SSR.
- **Opción A (React):** el frontend funciona como CSR. El estudiante debe explicar qué ventajas y limitaciones tiene ese enfoque para su proyecto.

5. Puntos adicionales (opcionales)

Estos puntos **suman a la nota**, pero el proyecto debe estar completo sin ellos.

5.1 Inteligencia artificial

Incorporar una funcionalidad con IA que aporte valor real al negocio elegido. Por ejemplo:

- Una **validación** inteligente de datos (revisar si un texto o formulario es coherente).

- Una **recomendación** tras analizar la información del tenant (resumir ventas, sugerir una acción, clasificar solicitudes).

Se puede usar el plan gratuito de Gemini. Tener en cuenta:

- Los límites son **estrictos y cambian por modelo**. Cada estudiante debe consultar sus límites reales en Google AI Studio antes de diseñar.
- Los límites se aplican **por proyecto de Google Cloud**, no por API key. Cada estudiante debe crear su propio proyecto.
- El nombre del modelo debe estar en una variable de entorno, porque los modelos cambian con frecuencia.
- Se valora que la funcionalidad use caché, aplique rate limiting propio y maneje el error 429 con un mensaje claro, sin romper la aplicación.

5.2 Automatización con n8n (o herramienta equivalente)

Integrar un flujo automatizado que reaccione a eventos del sistema. Por ejemplo:

- Enviar un correo o un mensaje cuando se crea un tenant nuevo.
- Generar un reporte periódico con los datos de un tenant.
- Notificar cuando ocurre un evento importante (una reserva, un pedido, una alerta).

Se debe explicar cómo se dispara el flujo (por ejemplo, mediante un webhook) y qué pasa si el flujo falla.

5.3 Otros puntos opcionales

- Despliegue en un servidor real accesible desde internet.
- Pruebas automatizadas de la lógica de aislamiento entre tenants.
- Roles dentro de cada tenant (administrador y usuario).

6. Temas a repasar antes de empezar

- **Arquitecturas multitenant:** base de datos por tenant, schema por tenant y tabla compartida con columna de tenant. Ventajas, límites y cuándo conviene cada una.
- **Resolución del tenant:** por subdominio, por header o por el contenido del JWT.
- **JWT y cookies httpOnly:** cómo se firman, qué contienen, expiración y seguridad.
- **ORM y migraciones:** cómo aplicar el mismo conjunto de tablas a varios schemas.
- **Idempotencia:** qué es, por qué los reintentos y los dobles clics la hacen necesaria, cómo se guarda una clave de idempotencia.
- **Rate limiting:** algoritmos principales, códigos de respuesta y headers.
- **Docker y Docker Compose:** servicios, redes, variables de entorno y volúmenes.

- **CI/CD:** etapas de un pipeline y cuándo debe fallar.
- **SSR, CSR y SSG:** diferencias y casos de uso de cada una.
- **Conexiones a Supabase:** diferencia entre la conexión directa y el pooler, y cuál conviene para migraciones y cuál para la aplicación en ejecución.

7. Cronograma sugerido

Es una guía, cada estudiante organiza su tiempo.

- **Día 1:** elegir el nicho, diseñar el modelo y levantar el entorno con Docker y Postgres.
- **Día 2:** autenticación y resolución del tenant.
- **Día 3:** creación automática del schema del tenant y sus tablas.
- **Día 4:** CRUD del dominio sobre el schema del tenant.
- **Día 5:** idempotencia y rate limiting.
- **Día 6:** CI/CD, documentación y, si da el tiempo, puntos adicionales.
- **Día 7:** pruebas finales y sustentación.

8. Entregables

- Repositorio con el código y un historial de commits coherente.
- `README.md` según la sección 4.8.
- `docker-compose.yml` y `.env.example`.
- Pipeline de CI/CD funcionando.
- Sustentación oral individual.

9. Criterios de evaluación

- **Multitenancy y aislamiento de datos:** 25 puntos
- **Autenticación y seguridad:** 15 puntos
- **Idempotencia:** 15 puntos
- **Rate limiting:** 15 puntos
- **Arquitectura, buenas prácticas y organización del código:** 10 puntos
- **Docker y CI/CD:** 10 puntos
- **Sustentación y documentación:** 10 puntos

Puntos adicionales: hasta +10 por la funcionalidad con IA y hasta +10 por la automatización.

10. Reglas

- El trabajo es **individual**.
- Se permite usar IA como apoyo. Todo lo que esté en el proyecto debe poder **explicarse** durante la sustentación.
- Durante la sustentación se pedirá explicar o modificar una parte del código en vivo, por ejemplo cómo se crea el schema de un tenant o cómo se elige la conexión en cada petición.
- No se deben subir claves, tokens ni contraseñas reales al repositorio.